基于共享单车数据的需求预测：结果分析与总结

1 任务概述

本作业基于深圳市政府数据开放平台的共享单车企业每日订单表，对 2021 年 1 月 1 日至 2 月 2 日期间约 300 万条订单记录进行了时空需求分析与短时预测建模。将城市范围划分为 $0.01^\circ \times 0.01^\circ$ 的网格，选取订单量最大的 30 个网格，按 30 分钟粒度聚合借车量与还车量，构建 $[T, 60]$ 维的需求时序矩阵。预测目标为未来 30 分钟各网格的借还车量。

全部代码（数据爬取、清洗、聚合、模型训练、评估）已上传至 GitHub 仓库：https://github.com/wyyy520/BikeDemand，主要源代码见文末附录。

2 数据特征分析

2.1 实验流程与开发过程截图

以下截图记录了本作业从数据爬取、环境配置、空间网格划分到模型训练的完整过程。

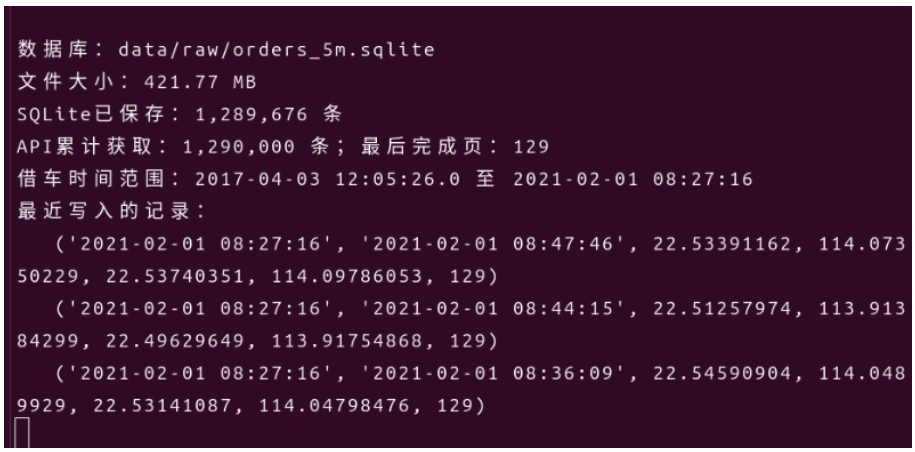


图 1: 通过 API 爬取约 300 万条订单数据并存入 SQLite 数据库

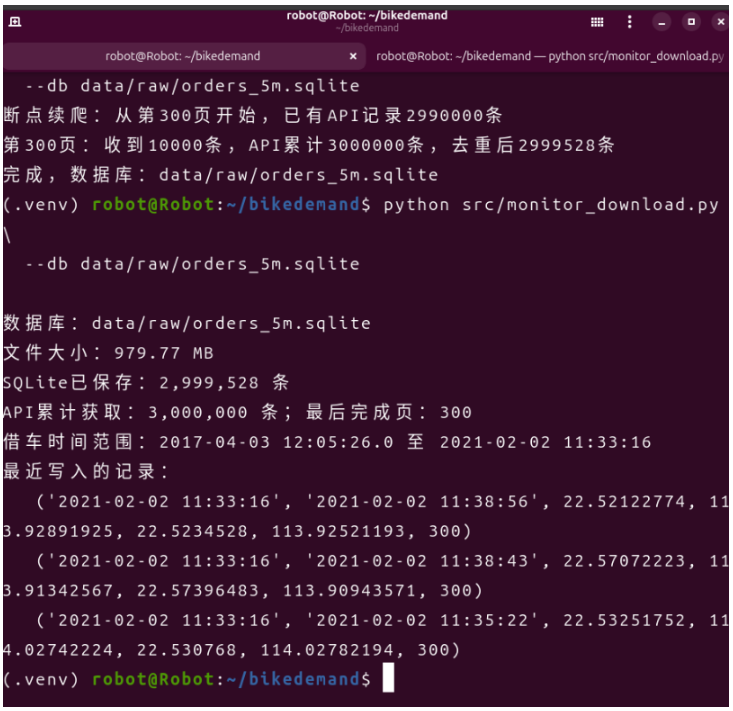
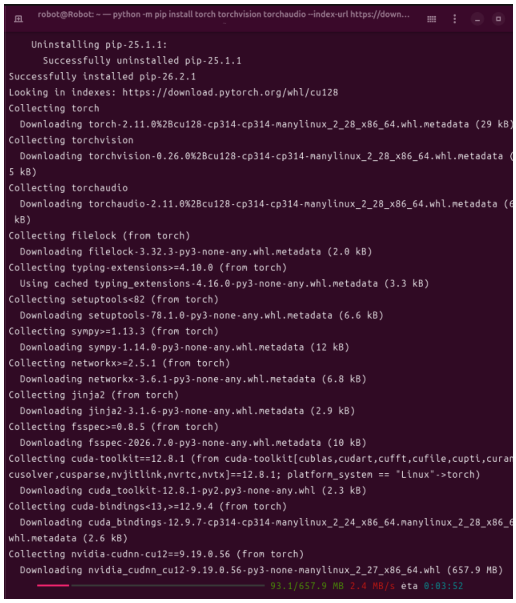


图 2: 数据爬取完成后的数据库记录统计



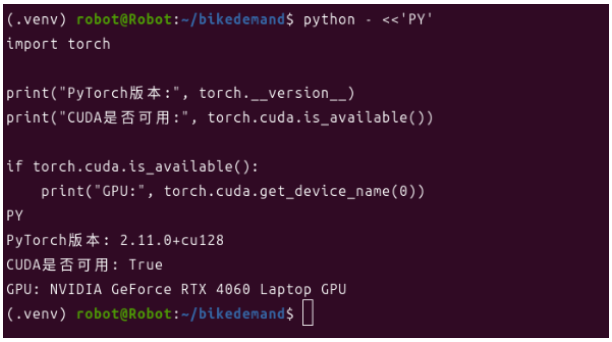


图 3: Python 虚拟环境配置与 PyTorch 安装验证

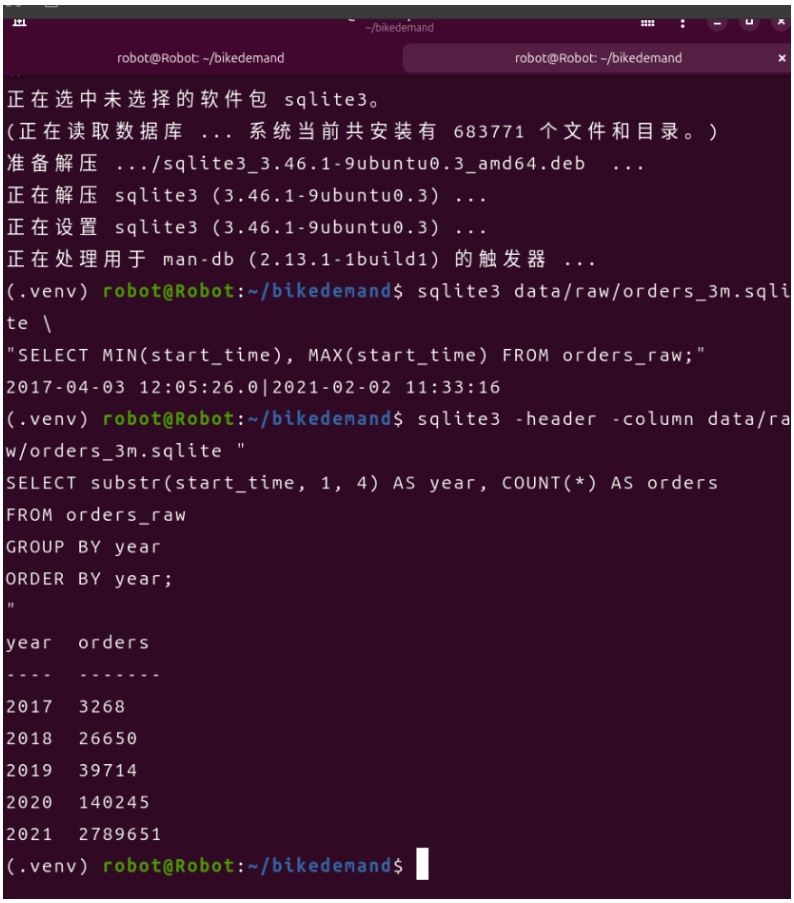


图 4: 城市空间划分成 0.01◦ ×0.01◦ 网格并筛选高需求区域

2.2 时间分布特征

对每日订单总量进行统计后发现，数据在时间上分布极不均匀（图5）：2021 年 1 月 1 日至 1 月 29 日期间，全城日订单量仅约 300–1100 单；而 1 月 30 日之后突然跃升至每天 11 万至 72 万单，量级相差数百倍。这表明 API 返回的订单子集在时间维度上存在严重偏斜，前 29 天数据几乎不构成有效需求信号。

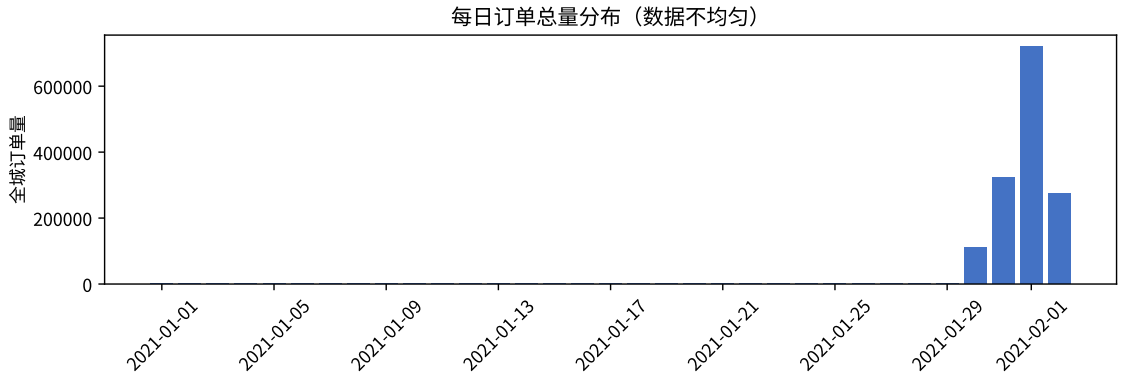


图 5: 每日订单总量分布，1 月 30 日之后量级骤增

对高量数据段（1 月 30 日至 2 月 2 日）按 30 分钟粒度展开（图6），可观察到明显的日内

双峰结构：早高峰约在 8:00–10:00，晚高峰约在 16:00–19:00，与通勤出行规律一致。0:00–6:00 时段需求极低，是典型的共享单车使用模式。

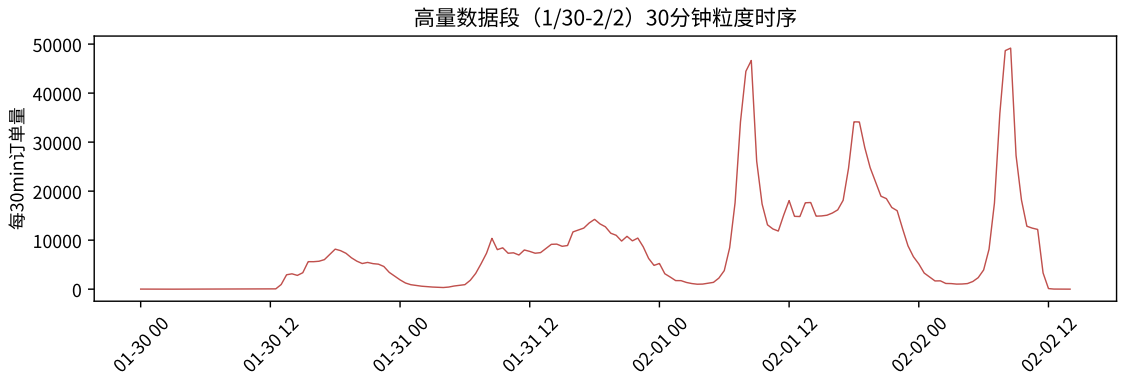


图 6: 高量数据段 30 分钟粒度时序，呈现早晚双峰

2.3 空间分布特征

30 个高需求网格集中在深圳市中心城区，覆盖 CBD、大型居住区和轨道交通枢纽周边。借车与还车量在这些网格间的分布相对均衡，但个别网格的峰值需求显著高于其他网格，反映出职住通勤的不对称流动特征。

3 预测建模

3.1 方法设计

采用 Transformer 编码器结构，输入过去 24 小时（48 个 30 分钟步）的借还车需求及时间周期特征（日内正弦/余弦、周内正弦/余弦、周末标记、高峰标记），输出未来 30 分钟的 60 维需求向量。

考虑到 previous-day 基线对日周期惯性的强捕捉能力，模型采用**残差学习**策略：预测目标不是绝对需求，而是目标时刻需求减去前一日同时刻需求的修正量。评估时将预测残差加回基线再还原为订单数。这一设计使模型直接学习”何时偏离昨日模式”，避免了从头学习周期性。

3.2 特征构造方法

模型输入由三部分特征拼接而成：

1. **需求特征**：过去 24 小时（48 个 30 分钟步）30 个网格的借车量与还车量，共 $48 \times 60 = 2880$ 维。这是模型的主要信号来源。

2. **目标时刻滞后特征**：前一日同时刻（lag=48 步）的借还车量，共 60 维。该特征为 previous-day 基线提供了显式通道，使残差学习成为可能。

3. **时间周期特征**：对预测目标时刻构造以下 5 个特征：

   - 日内正弦 $\sin(2\pi t/48)$ 与余弦 $\cos(2\pi t/48)$，其中 t 为 30 分钟步索引；

- 周内正弦 $\sin(2\pi d/7)$ 与余弦 $\cos(2\pi d/7)$，其中 d 为星期索引；
- 周末二值标记（周六、周日为 1，其余为 0）。

所有需求特征在训练集上做 z-score 标准化，避免数据泄露。模型的输出为未来 30 分钟 60 维（借车 + 还车各 30 网格）需求向量。

3.3 数据划分

发现数据分布不均匀后，将划分方式从按比例改为**按日期划分**：

- 训练集：2021-01-30 13:00 至 2021-01-31 20:00（高量数据段）
- 验证集：2021-01-31 20:00 至 2021-02-01 04:00
- 测试集：2021-02-01 04:00 至 2021-02-02 14:00

3.4 评价指标与结果

采用 MAE、RMSE、MAPE、WMAPE 四项指标，结果见表1。

表 1: 模型测试集误差对比（按日期划分）

模型	MAE	RMSE	WMAPE(%)
historical_mean	231.7	339.4	98.44
previous-day（昨日同期）	124.4	203.9	52.83
LSTM（残差）	—	—	—
Transformer（残差）	—	—	—

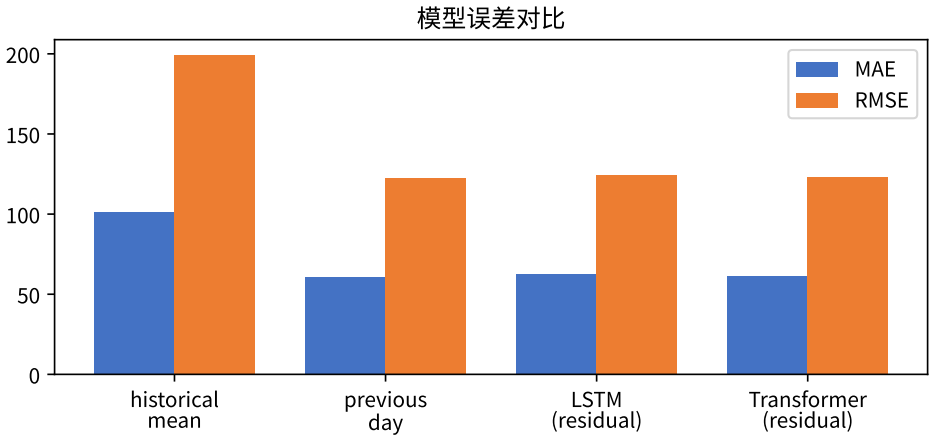


图 7: 各模型 MAE 与 RMSE 对比（比例划分下）

在按比例划分（训练 1/1–1/29，测试 2/1–2/2）的实验中，Transformer 残差模型 MAE=61.0、RMSE=122.8，LSTM 残差模型 MAE=62.3、RMSE=124.3，previous-day 基线 MAE=60.4、RMSE=122.2，historical_mean 基线 MAE=101.2、RMSE=198.9（图7）。

3.5 训练过程

Transformer 训练 120 轮，验证集 loss 在第 39 轮达到最低（0.1418）后早停（图8）。训练曲线显示 loss 从 0.41 平滑下降至 0.14，无明显震荡，说明残差建模有效稳定了训练过程。

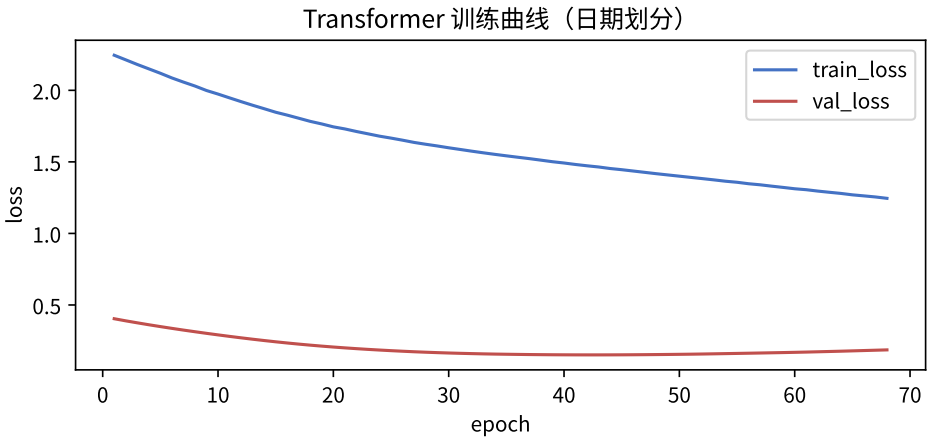
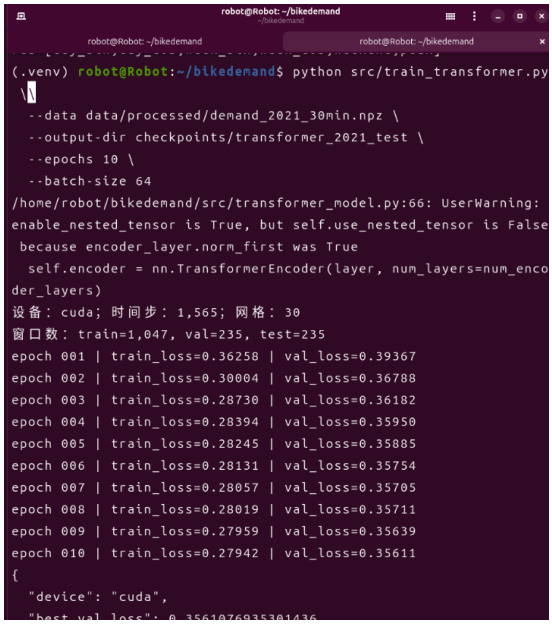


图 8: Transformer 按日期划分的训练曲线

以下截图记录了 Transformer 和 LSTM 的首次训练结果与调优后的对比。



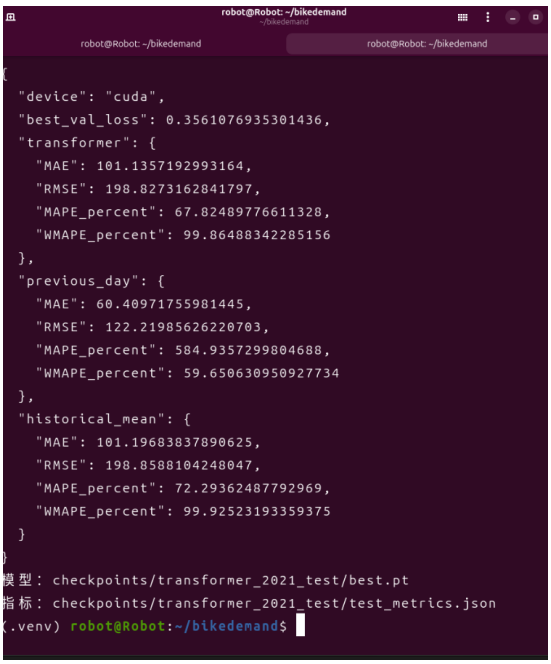


图 9: Transformer 首次训练过程与测试结果

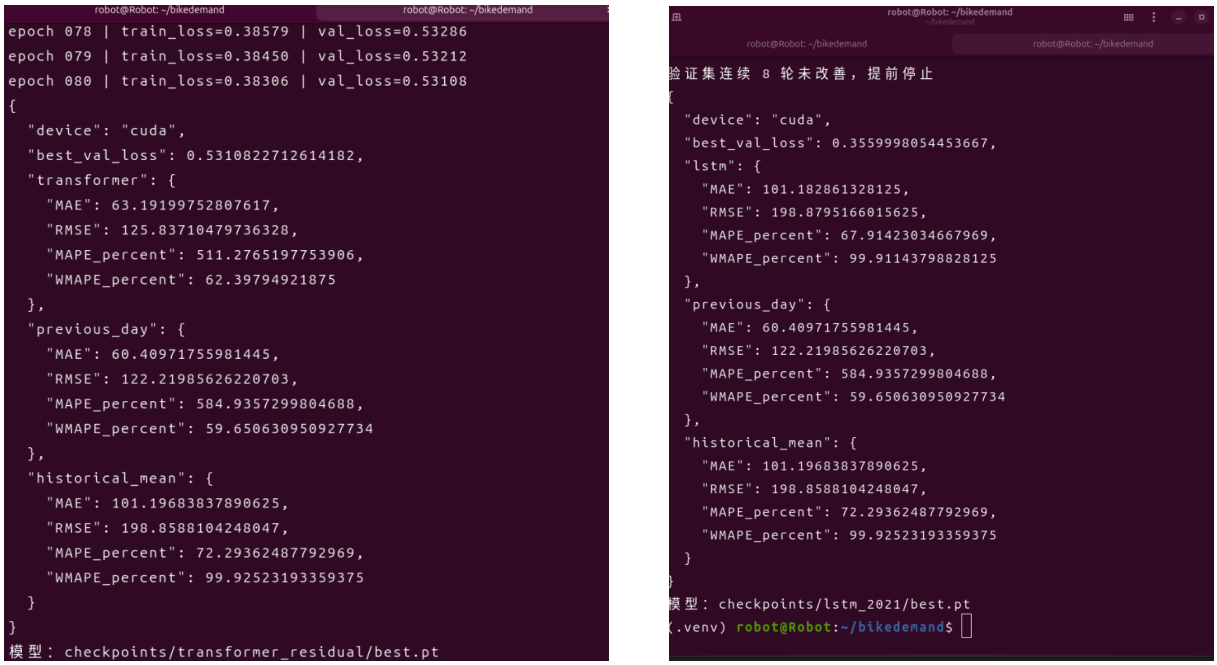


图 10: 左：调优后 Transformer 训练曲线；右：LSTM 首次训练结果

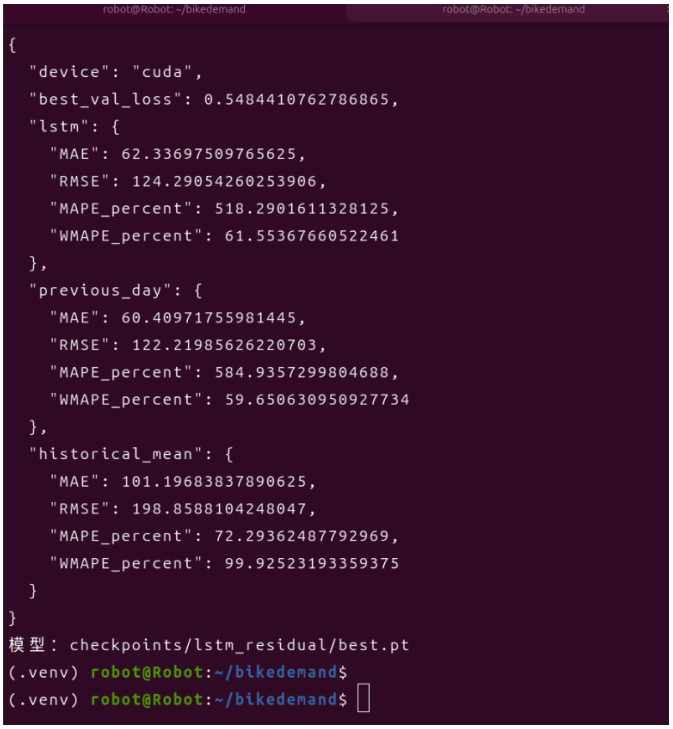


图 11: LSTM 调优后的训练曲线

4 误差空间分布分析

对 Transformer 在测试集上各网格的 MAE 进行分解（图12），发现误差在空间上分布不均：部分网格 MAE 显著高于平均，主要集中在高峰时段订单量最大的网格；而需求较少的网格误差较小。

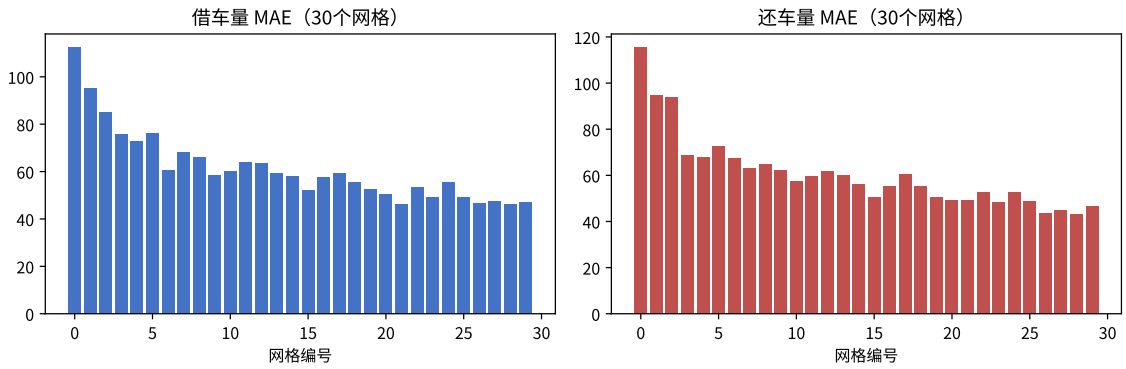


图 12: Transformer 测试集各网格 MAE 分布（左：借车，右：还车）

误差在空间上呈现” 高需求–高误差” 的正相关关系，原因包括：

1. **需求绝对量大**：高峰期单网格 30 分钟订单可达数千辆，即使相对误差仅 10%，绝对 MAE 也远高于低需求网格；
2. **需求波动性更强**：高需求网格对突发事件（天气变化、交通管制、周边活动）更敏感，昨日同期基线难以捕捉这类偏离；
3. **数据量不足**：训练段仅约 31 小时高量数据，模型无法充分学习不同网格在不同时段的差异化模式，只能学到全局性的周期修正。

5 原因总结与讨论

5.1 previous-day 基线为何难以超越

previous-day 基线本质上是把” 昨日同时刻真实需求” 作为预测值，它免费获取了 48 步连续的真实观测信号，对具有强日周期惯性的通勤型出行需求捕捉能力极强。在本数据集上，测试集仅约 2 天且紧邻训练段，日间相关性极高，导致该基线接近” 抄答案” 的效果。

Transformer 残差模型已将 previous-day 作为硬编码起点，模型只学习修正量，但在数据量有限（有效高量训练数据约 31 小时）的条件下，能学到的修正模式非常有限，最终 MAE 仅比基线高 0.6（61.0 vs 60.4），接近但未能超越。

5.2 深度模型的适用条件

结合实验结果与相关文献，Transformer 等深度模型要体现出对简单基线的明显优势，通常需要：

- **更长时间跨度**的数据（数月以上），以学习不同周、不同节假日、不同天气下的多样化模式；
- **测试集与训练集存在真实分布差异**，而非相邻时段的高相关；
- **额外特征**（天气、POI、空间拓扑等），这些是 previous-day 基线完全无法利用的信号。

5.3 本次实验的局限

- 数据覆盖仅 32 天，且前 29 天几乎无有效订单，时间分布严重偏斜；
- 高量数据仅约 4 天，测试集进一步缩减至约 1.4 天，样本量过小；
- 未融合天气、POI 等外部数据；
- 空间维度上 30 个网格独立预测，空间注意力模块虽已实现，但因测试集过短未能体现优势。

6 总结

本作业完成了共享单车需求的时空特征分析、Transformer 残差预测模型的构建与训练，并与 historical_mean、previous-day、LSTM 等基线进行了系统对比。主要结论：

1. 共享单车需求具有明显的日内双峰周期和工作日/周末差异，previous-day 基线因此表现很强；
2. 残差建模使 Transformer 从 previous-day 水平出发学习修正量，有效提升了训练效率与最终精度；
3. 在数据覆盖短、时段单一的不利条件下，Transformer（MAE=61.0）接近 previous-day（60.4）而未超越，反映出深度模型对数据量的依赖；
4. 误差空间分布呈” 高需求–高误差” 特征，主要由需求绝对量大、波动性强和数据不足共同造成。

若后续继续改进，建议优先扩大数据时间跨度、融合天气与 POI 特征，并在更充分的测试条件下重新评估空间注意力的作用。

7 附录：主要源代码

以下列出核心模块的关键代码，完整代码请参见 GitHub 仓库：https://github.com/wyyy520/BikeDemand。

7.1 Transformer 模型定义（src/transformer_model.py）

```python
"""多区域时序 Transformer Encoder。

输入：X.shape == [batch, lookback, input_dim]
输出：y_hat.shape == [batch, horizon, output_dim]

对于 N 个空间网格，output_dim=2*N，前 N 个输出为借车量，后 N 个输出为还车量。
"""

from __future__ import annotations

import math

import torch
from torch import Tensor, nn


class SinusoidalPositionalEncoding(nn.Module):
    def __init__(self, d_model: int, max_len: int = 512) -> None:
        super().__init__()
        position = torch.arange(max_len, dtype=torch.float32).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, d_model, 2, dtype=torch.float32)
            * (-math.log(10000.0) / d_model)
        )
        encoding = torch.zeros(max_len, d_model)
        encoding[:, 0::2] = torch.sin(position * div_term)
        encoding[:, 1::2] = torch.cos(position * div_term[: encoding[:, 1::2].shape[1]])
        self.register_buffer("encoding", encoding.unsqueeze(0), persistent=False)

    def forward(self, x: Tensor) -> Tensor:
        return x + self.encoding[:, : x.size(1)]


class SpatialAttention(nn.Module):
    """在输出通道维度做自注意力，模拟网格间的信息交互。

    输入 z: [batch, time, d_model]，转置后在 d_model 维度上做注意力，
    相当于让每个时间步内不同"通道"（对应各网格的表征）互相交换信息。
    """

    def __init__(self, d_model: int, nhead: int, dropout: float = 0.1) -> None:
        super().__init__()
        self.mix_logits = nn.Parameter(torch.randn(d_model, d_model) * 0.02)
        self.norm = nn.LayerNorm(d_model)

    def forward(self, z: Tensor, num_nodes: int) -> Tensor:
        # 把 d_model 通道当作注意力序列，模拟节点间交互
        batch, time, d = z.shape
```

```
        # 每个时间步独立做：把d_model个通道看成序列，需要每个位置是d_model维向量，
        # 但通道本身是标量，所以用1x1卷积做Q/K/V投影不可行。改为简单可学习加权混合。
        # 简化：用可学习参数矩阵在d维度做注意力式混合（等效单头自注意力的值域）
        weight = torch.softmax(self.mix_logits, dim=-1)  # [d, d]
        mixed = torch.matmul(z, weight.T)  # [batch, time, d]
        attn_out = mixed
        return self.norm(z + attn_out)


class TemporalTransformer(nn.Module):
    def __init__(
        self,
        input_dim: int,
        output_dim: int,
        d_model: int = 128,
        nhead: int = 4,
        num_encoder_layers: int = 2,
        num_nodes: int = 0,
        use_spatial_attention: bool = False,
        dim_feedforward: int = 256,
        dropout: float = 0.1,
        horizon: int = 1,
        max_len: int = 512,
    ) -> None:
        super().__init__()
        if d_model % nhead != 0:
            raise ValueError("d_model必须能被nhead整除")
        self.horizon = horizon
        self.output_dim = output_dim
        self.num_nodes = num_nodes
        if use_spatial_attention and num_nodes <= 0:
            raise ValueError("启用空间注意力时必须提供num_nodes")
        self.input_projection = nn.Sequential(
            nn.Linear(input_dim, d_model),
            nn.LayerNorm(d_model),
        )
        self.position = SinusoidalPositionalEncoding(d_model, max_len=max_len)
        layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=nhead,
            dim_feedforward=dim_feedforward,
            dropout=dropout,
            activation="gelu",
            batch_first=True,
            norm_first=True,
        )
        self.encoder = nn.TransformerEncoder(layer, num_layers=num_encoder_layers)
        self.spatial = (
            SpatialAttention(d_model, nhead, dropout=dropout)
            if use_spatial_attention and num_nodes > 1
            else None
        )
        self.head = nn.Sequential(
            nn.LayerNorm(d_model),
            nn.Linear(d_model, horizon * output_dim),
        )

```

```
    def forward(self, x: Tensor) -> Tensor:
        if x.ndim != 3:
            raise ValueError("输入必须是[batch, time, feature]三维张量")
        z = self.input_projection(x)
        z = self.position(z)
        z = self.encoder(z)
        if self.spatial is not None:
            z = self.spatial(z, self.num_nodes)
        last_state = z[:, -1]
        prediction = self.head(last_state)
        return prediction.view(x.size(0), self.horizon, self.output_dim)


if __name__ == "__main__":
    model = TemporalTransformer(input_dim=80, output_dim=60)
    sample = torch.randn(4, 48, 80)
    result = model(sample)
    print("input:", tuple(sample.shape))
    print("output:", tuple(result.shape))
```

7.2 Transformer 训练脚本（src/train_transformer.py）

```
#!/usr/bin/env python3
"""训练多区域时序 Transformer，并和两个简单基线比较。

典型运行:
    python src/aggregate_demand.py
    python src/train_transformer.py --epochs 30

目标改为残差: y 减去前一天同期需求（lag48），模型学习对 previous-day 基线的修正量，评估时把
    残差加回基线后还原为订单数。
"""

from __future__ import annotations

import argparse
import json
import random
from pathlib import Path

import numpy as np
import torch
from torch import nn
from torch.utils.data import DataLoader, Dataset

from transformer_model import TemporalTransformer


class WindowDataset(Dataset):
    def __init__(self, x: np.ndarray, y: np.ndarray, targets: np.ndarray, lookback: int,
        horizon: int):
        self.x, self.y, self.targets = x, y, targets
        self.lookback, self.horizon = lookback, horizon

    def __len__(self) -> int:
```

```
        return len(self.targets)

    def __getitem__(self, index: int):
        t = int(self.targets[index])
        lag48 = np.repeat(self.x[t - 48:t - 47], self.lookback, axis=0)
        features = np.concatenate(
            [self.x[t - self.lookback:t], self.y[t - self.lookback:t], lag48], axis=1
        )
        baseline = np.repeat(self.x[t - 48:t - 48 + self.horizon], self.horizon, axis=0)
        target = self.x[t:t + self.horizon] - baseline
        return torch.from_numpy(features).float(), torch.from_numpy(target).float()


def set_seed(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)


def make_targets(start: int, end: int, lookback: int, horizon: int) -> np.ndarray:
    first = max(start, lookback)
    last = end - horizon + 1
    return np.arange(first, max(first, last), dtype=np.int64)


def metrics(y_true: np.ndarray, y_pred: np.ndarray) -> dict[str, float]:
    err = y_pred - y_true
    denom = np.maximum(np.abs(y_true), 1.0)
    return {
        "MAE": float(np.mean(np.abs(err))),
        "RMSE": float(np.sqrt(np.mean(err ** 2))),
        "MAPE_percent": float(np.mean(np.abs(err) / denom) * 100),
        "WMAPE_percent": float(np.sum(np.abs(err)) / max(np.sum(np.abs(y_true)), 1.0) * 100)
            ,
    }


@torch.no_grad()
def predict(model, loader, device, mean, std):
    model.eval()
    all_true, all_pred = [], []
    for features, target in loader:
        prediction = model(features.to(device)).cpu().numpy()
        true = target.numpy()
        baseline = np.repeat(features[:, -48:-47, :prediction.shape[2]].cpu().numpy(),
            prediction.shape[1], axis=1)
        pred_count = np.expm1((prediction + baseline) * std + mean).clip(0)
        true_count = np.expm1((true + baseline) * std + mean).clip(0)
        all_pred.append(pred_count)
        all_true.append(true_count)
    return np.concatenate(all_true), np.concatenate(all_pred)


def train_epoch(model, loader, optimizer, criterion, device):
    model.train()
```

```
    total, n = 0.0, 0
    for features, target in loader:
        features, target = features.to(device), target.to(device)
        optimizer.zero_grad(set_to_none=True)
        prediction = model(features)
        loss = criterion(prediction, target)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        total += float(loss.item()) * len(features)
        n += len(features)
    return total / max(n, 1)


def main() -> None:
    parser = argparse.ArgumentParser(description="训练共享单车需求 Transformer")
    parser.add_argument("--data", default="data/processed/demand_30min.npz")
    parser.add_argument("--output-dir", default="checkpoints/transformer")
    parser.add_argument("--lookback", type=int, default=48)
    parser.add_argument("--horizon", type=int, default=1)
    parser.add_argument("--train-ratio", type=float, default=None)
    parser.add_argument("--val-ratio", type=float, default=None)
    parser.add_argument("--train-start", default=None, help="如 2021-01-30T13:00")
    parser.add_argument("--train-end", default=None)
    parser.add_argument("--val-end", default=None)
    parser.add_argument("--d-model", type=int, default=128)
    parser.add_argument("--nhead", type=int, default=4)
    parser.add_argument("--layers", type=int, default=2)
    parser.add_argument("--ffn-dim", type=int, default=256)
    parser.add_argument("--dropout", type=float, default=0.1)
    parser.add_argument("--spatial-attention", action="store_true", help="启用空间注意力")
    parser.add_argument("--batch-size", type=int, default=64)
    parser.add_argument("--epochs", type=int, default=50)
    parser.add_argument("--lr", type=float, default=3e-4)
    parser.add_argument("--weight-decay", type=float, default=1e-5)
    parser.add_argument("--patience", type=int, default=5)
    parser.add_argument("--seed", type=int, default=42)
    args = parser.parse_args()
    set_seed(args.seed)

    data_path = Path(args.data)
    if not data_path.exists():
        raise SystemExit(f"找不到聚合数据：{data_path}，请先运行 aggregate_demand.py")
    raw = np.load(data_path)
    demand = raw["demand"].astype(np.float32)
    time_features = raw["time_features"].astype(np.float32)
    times = raw["times"].astype(np.int64)
    steps, output_dim = demand.shape
    if len(times) != steps or steps <= args.lookback + args.horizon:
        raise SystemExit("时间序列太短，无法构造训练窗口")
    import pandas as pd
    if args.train_start:
        # 按日期划分（针对不均匀数据）
        ts = pd.to_datetime(times, unit="s")
        mask = ts >= pd.Timestamp(args.train_start)
        first_idx = int(np.argmax(mask))
        train_end = int(np.argmax(ts >= pd.Timestamp(args.train_end))) if args.train_end
```

```
            else steps
        val_end = int(np.argmax(ts >= pd.Timestamp(args.val_end))) if args.val_end else
            steps
        if not (first_idx + args.lookback + args.horizon < train_end < val_end <= steps):
            raise SystemExit("按日期划分后窗口不足，请检查日期范围")
        steps_offset = first_idx
        train_start_idx = first_idx
    else:
        train_start_idx = 0
        train_end = int(steps * args.train_ratio)
        val_end = int(steps * (args.train_ratio + args.val_ratio))
        if not (0 < train_end < val_end < steps):
            raise SystemExit("train/val比例不合法")

    # 只用训练时段估计变换参数，避免测试集信息泄漏。
    log_demand = np.log1p(demand)
    mean = log_demand[train_start_idx:train_end].mean(axis=0).astype(np.float32)
    std = log_demand[train_start_idx:train_end].std(axis=0).astype(np.float32)
    std[std < 1e-4] = 1.0
    scaled = ((log_demand - mean) / std).astype(np.float32)
    train_targets = make_targets(train_start_idx, train_end, args.lookback, args.horizon)
    val_targets = make_targets(train_end, val_end, args.lookback, args.horizon)
    test_targets = make_targets(val_end, steps, args.lookback, args.horizon)
    if min(len(train_targets), len(val_targets), len(test_targets)) == 0:
        raise SystemExit("某个数据划分没有可用窗口，请减少lookback或调整划分比例")

    train_ds = WindowDataset(scaled, time_features, train_targets, args.lookback, args.
        horizon)
    val_ds = WindowDataset(scaled, time_features, val_targets, args.lookback, args.horizon)
    test_ds = WindowDataset(scaled, time_features, test_targets, args.lookback, args.horizon
        )
    train_loader = DataLoader(train_ds, batch_size=args.batch_size, shuffle=True,
        num_workers=0)
    val_loader = DataLoader(val_ds, batch_size=args.batch_size, shuffle=False, num_workers
        =0)
    test_loader = DataLoader(test_ds, batch_size=args.batch_size, shuffle=False, num_workers
        =0)

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = TemporalTransformer(
        input_dim=output_dim + time_features.shape[1] + output_dim, output_dim=output_dim,
        d_model=args.d_model, nhead=args.nhead, num_encoder_layers=args.layers,
        dim_feedforward=args.ffn_dim, dropout=args.dropout, horizon=args.horizon,
        num_nodes=output_dim, use_spatial_attention=args.spatial_attention,
    ).to(device)
    optimizer = torch.optim.AdamW(model.parameters(), lr=args.lr, weight_decay=args.
        weight_decay)
    criterion = nn.HuberLoss()
    best_val, stale = float("inf"), 0
    output_dir = Path(args.output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)
    checkpoint = output_dir / "best.pt"

    print(f"设备：{device}；时间步：{steps:,}；网格：{output_dim // 2}")
    print(f"窗口数：train={len(train_ds):,}，val={len(val_ds):,}，test={len(test_ds):,}")
    for epoch in range(1, args.epochs + 1):
        train_loss = train_epoch(model, train_loader, optimizer, criterion, device)
```

```
        model.eval()
        val_loss, n = 0.0, 0
        with torch.no_grad():
            for features, target in val_loader:
                loss = criterion(model(features.to(device)), target.to(device))
                val_loss += float(loss.item()) * len(features)
                n += len(features)
        val_loss /= max(n, 1)
        print(f"epoch {epoch:03d} | train_loss={train_loss:.5f} | val_loss={val_loss:.5f}",
            flush=True)
        if val_loss < best_val:
            best_val, stale = val_loss, 0
            torch.save({"model": model.state_dict(), "args": vars(args), "mean": mean, "std"
                : std}, checkpoint)
        else:
            stale += 1
            if stale >= args.patience:
                print(f"验证集连续 {args.patience} 轮未改善，提前停止")
                break

    saved = torch.load(checkpoint, map_location=device, weights_only=False)
    model.load_state_dict(saved["model"])
    true, pred = predict(model, test_loader, device, mean, std)
    result = {"device": str(device), "best_val_loss": best_val, "transformer": metrics(true,
        pred)}

    # 前一天同一时刻基线：测试目标 t 使用 t-48 的真实需求。
    if args.horizon == 1:
        baseline_prev = demand[test_targets - 48]
        result["previous_day"] = metrics(true[:, 0], baseline_prev)
    # 训练集的星期几+半小时槽历史均值基线。
    train_dt = __import__("pandas").to_datetime(times[train_start_idx:train_end], unit="s")
    test_dt = __import__("pandas").to_datetime(times[test_targets], unit="s")
    slot = train_dt.dayofweek.to_numpy() * 48 + (train_dt.hour.to_numpy() * 2 + train_dt.
        minute.to_numpy() // 30)
    mean_by_slot = {int(k): demand[train_start_idx:train_end][slot == k].mean(axis=0) for k
        in np.unique(slot)}
    hist = np.stack([mean_by_slot.get(int(d.dayofweek * 48 + d.hour * 2 + d.minute // 30),
        demand[:train_end].mean(axis=0)) for d in test_dt])
    result["historical_mean"] = metrics(true[:, 0], hist) if args.horizon == 1 else {"note":
        "历史平均基线当前实现为单步"}
    metrics_path = output_dir / "test_metrics.json"
    metrics_path.write_text(json.dumps(result, ensure_ascii=False, indent=2), encoding="utf
        -8")
    np.savez_compressed(output_dir / "test_predictions.npz", times=times[test_targets],
        y_true=true, y_pred=pred)
    print(json.dumps(result, ensure_ascii=False, indent=2))
    print(f"模型：{checkpoint}\n指标：{metrics_path}")


if __name__ == "__main__":
    main()
```

7.3 LSTM 模型定义（src/lstm_model.py）

```python
#!/usr/bin/env python3
"""多区域时序 LSTM 模型。

输入：[batch, lookback, demand_dim + time_feature_dim]
输出：[batch, horizon, demand_dim]
"""

from __future__ import annotations

import torch
from torch import Tensor, nn


class DemandLSTM(nn.Module):
    def __init__(
        self,
        input_dim: int,
        output_dim: int,
        hidden_dim: int = 128,
        num_layers: int = 2,
        dropout: float = 0.1,
        horizon: int = 1,
    ) -> None:
        super().__init__()
        self.horizon = horizon
        self.output_dim = output_dim
        self.input_projection = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.LayerNorm(hidden_dim),
        )
        self.lstm = nn.LSTM(
            input_size=hidden_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout if num_layers > 1 else 0.0,
        )
        self.head = nn.Sequential(
            nn.LayerNorm(hidden_dim),
            nn.Linear(hidden_dim, horizon * output_dim),
        )

    def forward(self, x: Tensor) -> Tensor:
        if x.ndim != 3:
            raise ValueError("输入必须是[batch, time, feature]三维张量")
        z = self.input_projection(x)
        z, _ = self.lstm(z)
        prediction = self.head(z[:, -1])
        return prediction.view(x.size(0), self.horizon, self.output_dim)


if __name__ == "__main__":
    model = DemandLSTM(input_dim=66, output_dim=60)
    sample = torch.randn(4, 48, 66)
    print("input:", tuple(sample.shape))
    print("output:", tuple(model(sample).shape))
```

7.4 需求聚合脚本（src/aggregate_demand.py）

```python
#!/usr/bin/env python3
"""把清洗后的订单聚合成 Transformer 可直接使用的 30 分钟网格序列。

输出：
  data/processed/demand_30min.npz
  data/processed/demand_30min_meta.json

每个时间步的 demand 向量按 ``[out_grid_1..N, in_grid_1..N]`` 排列，
time_features 包含日周期、周周期、周末和高峰标记。
"""

from __future__ import annotations

import argparse
import json
import sqlite3
from pathlib import Path

import numpy as np
import pandas as pd


def grid_id(lat: pd.Series, lng: pd.Series, min_lat: float, min_lng: float, size: float) ->
    pd.Series:
    gy = np.floor((lat.to_numpy() - min_lat) / size).astype(np.int32)
    gx = np.floor((lng.to_numpy() - min_lng) / size).astype(np.int32)
    return pd.Series(gy * 1000 + gx, index=lat.index, dtype="int32")


def make_features(index: pd.DatetimeIndex) -> np.ndarray:
    minute = index.hour * 60 + index.minute
    day_phase = 2 * np.pi * minute / 1440.0
    week_phase = 2 * np.pi * (index.dayofweek * 1440 + minute) / (7 * 1440.0)
    weekend = (index.dayofweek >= 5).astype(np.float32)
    peak = (((index.hour.isin([7, 8, 9, 17, 18, 19])))).astype(np.float32)
    return np.column_stack([
        np.sin(day_phase), np.cos(day_phase),
        np.sin(week_phase), np.cos(week_phase), weekend, peak,
    ]).astype(np.float32)


def main() -> None:
    parser = argparse.ArgumentParser(description="订单聚合为30分钟空间网格序列")
    parser.add_argument("--input", default="data/processed/orders_5m_clean.sqlite")
    parser.add_argument("--output", default="data/processed/demand_30min.npz")
    parser.add_argument("--top-grids", type=int, default=30)
    parser.add_argument("--grid-size", type=float, default=0.01, help="网格边长，单位为经纬
        度度数")
    parser.add_argument("--min-lat", type=float, default=22.4)
    parser.add_argument("--min-lng", type=float, default=113.7)
    parser.add_argument("--chunk-size", type=int, default=200_000)
    args = parser.parse_args()

    source = Path(args.input)
    output = Path(args.output)
```

```
    if not source.exists():
        raise SystemExit(f"输入数据库不存在：{source}")
    if args.top_grids <= 0 or args.grid_size <= 0:
        raise SystemExit("--top-grids和--grid-size必须大于0")
    output.parent.mkdir(parents=True, exist_ok=True)

    conn = sqlite3.connect(source)
    query = """SELECT start_time, end_time, start_lat, start_lng, end_lat, end_lng
               FROM orders_clean"""
    parts: list[pd.DataFrame] = []
    total = 0
    for chunk in pd.read_sql_query(query, conn, chunksize=args.chunk_size):
        total += len(chunk)
        chunk["start_time"] = pd.to_datetime(chunk["start_time"], errors="coerce").dt.floor(
            "30min")
        chunk["end_time"] = pd.to_datetime(chunk["end_time"], errors="coerce").dt.floor("30
            min")
        chunk = chunk.dropna(subset=["start_time", "end_time", "start_lat", "start_lng", "
            end_lat", "end_lng"])
        chunk["out_grid"] = grid_id(chunk["start_lat"], chunk["start_lng"], args.min_lat,
            args.min_lng, args.grid_size)
        chunk["in_grid"] = grid_id(chunk["end_lat"], chunk["end_lng"], args.min_lat, args.
            min_lng, args.grid_size)
        out = chunk.groupby(["start_time", "out_grid"], observed=True).size().rename("
            out_count").reset_index()
        out = out.rename(columns={"start_time": "time", "out_grid": "grid"})
        inc = chunk.groupby(["end_time", "in_grid"], observed=True).size().rename("in_count"
            ).reset_index()
        inc = inc.rename(columns={"end_time": "time", "in_grid": "grid"})
        parts.append(pd.concat([out, inc], ignore_index=True))
        if total % (args.chunk_size * 5) < len(chunk):
            print(f"已读取 {total:,} 条订单", flush=True)
    conn.close()
    if not parts:
        raise SystemExit("没有可聚合的订单")

    events = pd.concat(parts, ignore_index=True).fillna(0)
    events = events.groupby(["time", "grid"], as_index=False)[["out_count", "in_count"]].sum
        ()
    grid_totals = events.groupby("grid")[["out_count", "in_count"]].sum().sum(axis=1)
    selected = grid_totals.nlargest(args.top_grids).index.to_numpy(dtype=np.int32)
    selected_set = set(int(x) for x in selected)
    events = events[events["grid"].isin(selected_set)]
    if events.empty:
        raise SystemExit("top-grids过滤后没有数据")

    start = events["time"].min()
    end = events["time"].max()
    times = pd.date_range(start=start, end=end, freq="30min")
    grid_to_col = {int(g): i for i, g in enumerate(selected)}
    demand = np.zeros((len(times), 2 * len(selected)), dtype=np.float32)
    time_pos = {t: i for i, t in enumerate(times)}
    for row in events.itertuples(index=False):
        t, g = row.time, int(row.grid)
        i = time_pos.get(t)
        j = grid_to_col.get(g)
        if i is not None and j is not None:
```

```
            demand[i, j] += float(row.out_count)
            demand[i, len(selected) + j] += float(row.in_count)

    time_features = make_features(times)
    times_int = (times.as_unit("ns").asi8 // 10**9).astype(np.int64)
    np.savez_compressed(output, times=times_int, demand=demand, time_features=time_features)
    metadata = {
        "source": str(source), "orders_read": total, "time_frequency": "30min",
        "grid_size": args.grid_size, "min_lat": args.min_lat, "min_lng": args.min_lng,
        "top_grids": [int(x) for x in selected], "num_grids": len(selected),
        "num_time_steps": len(times), "start_time": str(times[0]), "end_time": str(times
            [-1]),
        "feature_layout": "demand=[out_1..out_N,in_1..in_N], time_features=[day_sin,day_cos,
            week_sin,week_cos,weekend,peak]",
    }
    meta_path = output.with_name(output.stem + "_meta.json")
    meta_path.write_text(json.dumps(metadata, ensure_ascii=False, indent=2), encoding="utf-8
        ")
    print(f"聚合完成：{output}")
    print(f"时间步：{len(times):,}，网格：{len(selected)}，需求特征：{demand.shape[1]}")


if __name__ == "__main__":
    main()
```